

Abrazo mortal

Descripción del concepto

El abrazo mortal, también llamado interbloqueo, *deadlock* o bloqueo mutuo, se define como el bloqueo perpetuo de un conjunto de procesos, o hilos, donde cada uno de los procesos del conjunto está bloqueado esperando por un evento, generalmente la liberación de algún recurso solicitado, que sólo puede generar otro proceso bloqueado del conjunto. Este bloqueo es permanente porque no puede producirse ninguno de los eventos esperados. Para este tipo de problema no existe una solución eficiente.

Condiciones para la ocurrencia de un abrazo mortal

Las condiciones para su ocurrencia fueron descritas por primera vez en 1971 en el artículo *System deadlocks* escrito por Edward G. Coffman, M. J. Elphick y A. Shoshani. Estas condiciones deben cumplirse de forma simultánea, no son totalmente independientes entre ellas y establecen que un abrazo mortal puede ocurrir sólo si se cumplen las siguientes condiciones:

Exclusión mutua

Significa que si un proceso posee un conjunto de recursos compartibles estos no pueden ser usados por otro proceso hasta haber tenido todos los recursos necesarios y haber podido usarlos.

Espera y retención

Mientras un proceso espera la asignación de algún recurso no libera otros recursos que ya tiene asignado.

Sin desalojo

Ningún proceso puede ser forzado a abandonar un recurso que retenga. Es decir que, si un proceso tiene tomados recursos y hay otros recursos que no se le pueden asignar, no se le pueden quitar los que ya tiene asignados.

Espera circular

Se dice que dos o más procesos de una cadena cerrada de procesos están en espera circular de recursos cuando cada proceso tiene asignado un recurso y espera por otro recurso que ya está tomado por el siguiente proceso de la cadena y éste a su vez espera la liberación de otro recurso que también está asignado.

En la mayoría de los casos, las tres primeras condiciones son bastante necesarias. Por ejemplo, la exclusión mutua hace falta para asegurar la consistencia de resultados y la integridad de la base de datos. De forma similar, la expulsión o apropiación no se puede aplicar arbitrariamente y, cuando se encuentran involucrados especialmente recursos de datos, debe estar acompañada de un mecanismo de recuperación y reanudación, que devuelva a un proceso y a sus recursos a un estado previo adecuado, desde el que el proceso pueda finalmente repetir sus acciones. Puede existir un abrazo mortal con estas tres condiciones, pero no puede existir con sólo estas tres condiciones. Para que se produzca un abrazo mortal, se necesita la cuarta condición.

Que el sistema trabaje con exclusión mutua, espera y retención y sin desalojo resulta necesario, pero no es suficiente, para que exista un abrazo mortal. Presentes las tres primeras condiciones, la espera circular se convierte en una consecuencia potencial de aquellas tres. Es decir, dado que se producen las tres primeras condiciones, puede ocurrir una secuencia de eventos que desemboque en una espera circular sin solución. Esta clase de espera circular es, evidentemente, la definición de abrazo mortal y no tiene solución porque se mantienen las tres primeras condiciones. En otras palabras, la unión de las cuatro condiciones constituye una condición necesaria y suficiente para el abrazo mortal.

Clasificación de los recursos

Los recursos se pueden clasificar en:

Reutilizables y consumibles

Un recurso reutilizable es aquél que puede ser usado con seguridad por un proceso y no se agota con el uso. Por el contrario, los recursos consumibles son aquellos que pueden ser creados, o producidos, y se caracterizan porque dejan de existir cuando son consumidos una vez que un proceso los usa. Cualquiera de estos recursos puede causar un abrazo mortal.

Compartidos y exclusivos

Un recurso es exclusivo cuando al ser usado por un proceso, ningún otro lo puede usar. Pero, algunos recursos pueden ser usados simultáneamente por varios procesos: son recursos compartidos. Mientras que los recursos de tipo compartido no se ven afectados por los interbloqueos, ya que los procesos que quieran usarlos podrán hacerlo inmediatamente sin posibilidad de quedarse bloqueados, ocurre todo lo contrario con los recursos exclusivos. Luego, las estrategias de tratamiento de interbloqueos no tratan este tipo de recursos y se concentran únicamente en los recursos reutilizables de uso exclusivo.

Únicos y múltiples

Se diferencian en que los primeros sólo presentan una instancia mientras que los segundos presentan varias instancias del recurso.

Expropiables y no expropiables

Un recurso expropiable es aquel que se le puede quitar al proceso que lo posee sin efectos dañinos. Mientras que, un recurso no expropiable es uno que no se puede quitar a su propietario actual sin provocar un error en la tarea. Como usualmente el abrazo mortal comprende a los recursos no expropiables, ya que, aquel posible abrazo mortal que pueda comprender a los recursos expropiables usualmente se lo puede solucionar mediante la reasignación de los recursos de un proceso a otro, resulta que todo análisis del tema se centra en los recursos no expropiables.

Grafo de Asignación de Recursos (GAR)

Un grafo de asignación de recursos G consiste en un conjunto de nodos N y un conjunto de aristas A : $G = \{N, A\}$. Es un grafo dirigido que representa el estado del sistema en lo referido a los recursos y los procesos, de forma tal que cada proceso y cada recurso se representa por un nodo o vértice.

El conjunto de nodos o vértices se descompone a su vez en dos subconjuntos disjuntos que se corresponden con los procesos y los recursos. Cada recurso tiene asociado un valor que representa cuántas unidades, o instancias, del recurso existen.

A su vez, el conjunto de aristas también se descompone en dos subconjuntos que se corresponden con los dos nodos antes planteados:

- Aristas de asignación que relacionan recursos con procesos: una arista entre un recurso R_j y un proceso P_i indica que el proceso tiene asignada una unidad de dicho recurso.
- Aristas de solicitud que relacionan procesos con recursos: una arista entre un proceso P_i y un recurso R_j indica que el proceso está esperando la concesión de una unidad de dicho recurso.

Como con toda modelización, para el grafo de asignación de recursos, también existen restricciones. En este caso las restricciones de coherencia se implementarían de la siguiente forma:

- Restricción de asignación: el número de aristas que salen de un recurso debe ser menor o igual que su inventario.
- Restricción de solicitud: se debe cumplir que por cada pareja proceso I y recurso J , el número de aristas de asignación que van de R_J a P_I más el número de aristas de solicitud que conectan P_I a R_J sea menor o igual que el número de instancias del recurso.

Las formas utilizadas para la representación de todos los elementos del grafo son las siguientes:

Elemento	Forma
Nodo proceso P	
Nodo recurso R	
Instancias del recurso	
Arista de asignación de 1 instancia del recurso R al proceso P	

Par ordenado: (R(1),P)	
Arista de solicitud de 1 instancia del recurso R por el proceso P Par ordenado: (P,R(1))	

Suponer como ejemplo que, en un sistema con 3 recursos: R (1 unidad), S (2 unidad) y T (1 unidades), se ejecutan tres procesos: A, B y C, que realizan la siguiente secuencia de peticiones:

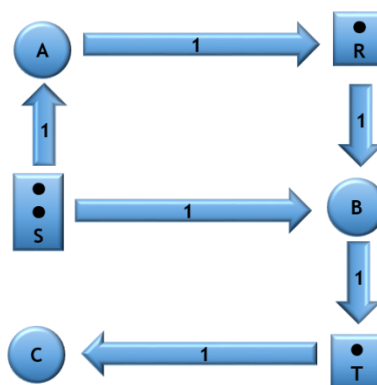
- Al proceso A se le asigna la unidad del recurso S que solicitó.
- Al proceso B se le asigna la unidad del recurso R y otra del S que pidió.
- Al proceso C se le asigna la unidad del recurso T que solicitó.
- El proceso A solicita 1 unidad del recurso R.
- El proceso B solicita 1 unidad del recurso T.

Los conjuntos que representan la situación del sistema después de ejecutarse esta secuencia es el siguiente:

$N = \{A, B, C, R(1), S(2), T(1)\}$

$A = \{(S(1), A), (R(1), B), (S(1), B), (T(1), C), (A, R(1)), (B, T(1))\}$

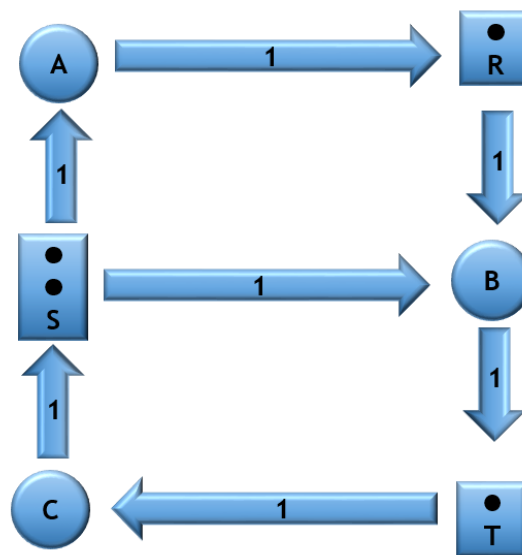
El grafo de asignación de recursos sería el siguiente:



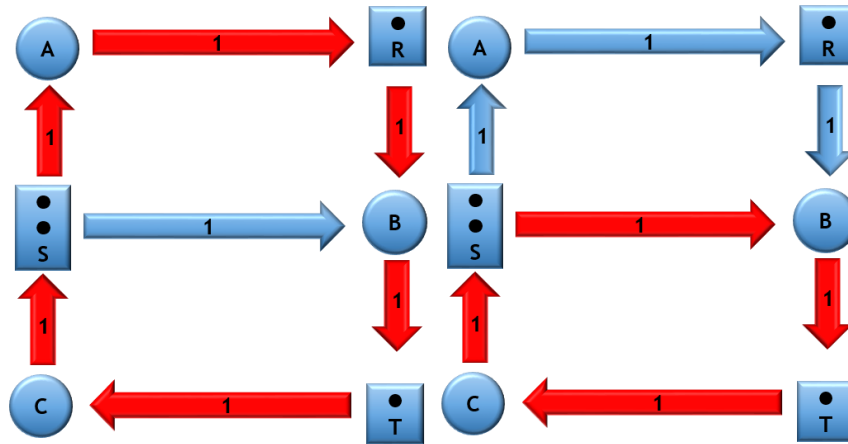
Analizando el grafo se puede decir que, si el grafo no tiene ciclo alguno no existe abrazo mortal, pero, si el grafo tiene un ciclo puede existir un abrazo mortal. ¿Cómo es posible esto? Si por cada recurso existe solo una instancia entonces, el que el grafo tenga un ciclo

implica sí o sí que existe abrazo mortal. Asimismo, si hay recursos con más de una instancia, pero, los recursos que conforman el ciclo tienen sólo una instancia, entonces los procesos que forman parte del ciclo están en un abrazo mortal. Luego la existencia de un ciclo en un grafo con esas características es condición necesaria y suficiente para que haya un abrazo mortal. Ahora bien, si existen varias instancias por tipo de recurso, la existencia de un ciclo no implica obligatoriamente la existencia de un abrazo mortal. Es decir que, el que haya un ciclo en el grafo es condición necesaria pero no suficiente para la aparición de un abrazo mortal.

Continuando con el ejemplo anterior, si el proceso C solicita una instancia del recurso S, se tendría el siguiente grafo:

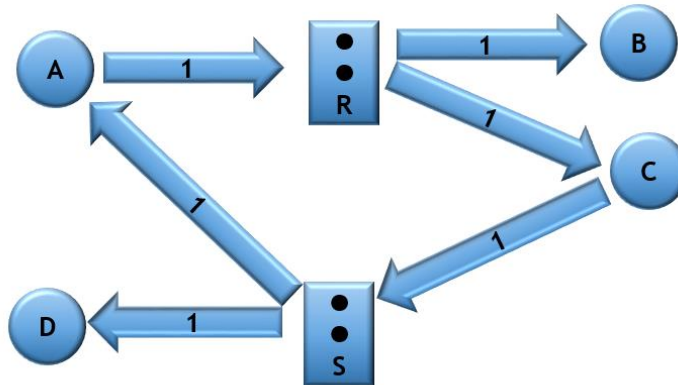


En el cual se tienen los siguientes ciclos:



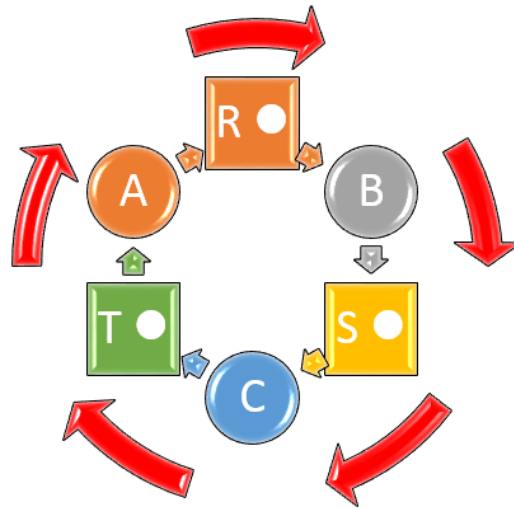
Donde los procesos A, B y C están en un abrazo mortal.

Sin embargo, en el siguiente grafo, que también presenta un ciclo:



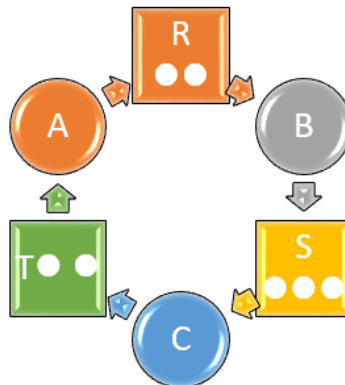
...no existe ningún abrazo mortal porque en algún momento el proceso D liberará la instancia del recurso S que tiene asignada haciendo desaparecer así el ciclo. En definitiva, si un grafo de asignación de recurso no tiene un ciclo, entonces el sistema no está en un abrazo mortal, mientras que, si tiene un ciclo, entonces el sistema puede o no estar en un abrazo mortal.

El siguiente grafo está mostrando un ejemplo de espera circular, condición imprescindible para que exista un abrazo mortal, marcando con las flechas en rojo el ciclo correspondiente:



En el mismo se tiene una única unidad de los recursos R, S y T. El proceso A retiene la unidad del recurso T y solicita una unidad del recurso R mientras que, el proceso B retiene la única unidad de R y pide una unidad de S, por último, el proceso C retiene la unidad del recurso S y solicita una unidad del recurso T.

Por el contrario, si se mantienen las mismas asignaciones y solicitudes, en el siguiente grafo no existirá abrazo mortal porque hay disponibles varias unidades, o instancias, de cada recurso:



Representación matricial

Además de la utilización de grafos, otra forma de representar el estado del sistema en lo referido a los recursos y a los procesos es mediante el uso de matrices y vectores. Siendo n el número de procesos existentes y m el número de recursos distintos que hay en el sistema entonces el significado de estas estructuras de datos es el siguiente:

- Matriz de asignación MA, de dimensión $n \times m$: en ella, el elemento MA_{IJ} especifica cuántas unidades del recurso J están asignadas al proceso I.
- Matriz de solicitud MS, de dimensión $n \times m$: aquí, cada elemento MS_{IJ} especifica cuántas unidades del recurso J está esperando el proceso I que se le concedan.
- Vector de recursos existentes VR, de dimensión m: en esta estructura, la componente VR_I especifica cuantas unidades del recurso I existen.

En este tipo de representación, las restricciones de coherencia del sistema implicarían lo siguiente:

- Restricción de asignación: se debe cumplir para cada recurso I que la suma de la columna J de la matriz MA ($\sum A_{IJ}$, para $I = 1, \dots, n$) sea menor o igual que el número de unidades existentes de dicho recurso (VR_J).
- Restricción de solicitud: se tiene que verificar para cada proceso I y recurso J la siguiente expresión $MA_{IJ} + MS_{IJ} \leq VR_J$.

A veces, a este conjunto de estructuras se le agrega un vector de recursos disponibles VD que refleja el número de unidades de cada recurso disponibles en un momento dado. Fácilmente se puede apreciar que este vector es innecesario, ya que su valor se puede deducir directamente a partir de la matriz de asignación MA y del vector de recursos existentes VR mediante el siguiente cálculo: $VD_J = VR_J - \sum MA_{IJ}$, para $J = 1, \dots, m$.

Ejemplo 1

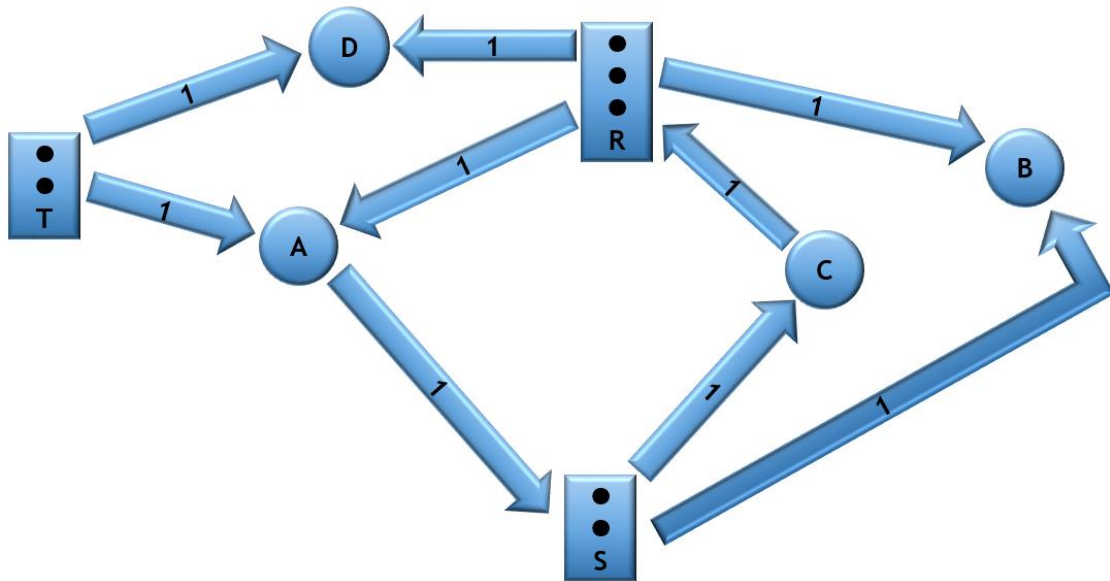
Dados los siguientes conjuntos de nodos (N) y aristas (AR)...

$N = \{A, B, C, D, R(3), S(2), T(2)\}$

$AR = \{(R(1), A), (R(1), B), (R(1), D), (S(1), B), (S(1), C), (T(1), A), (T(1), D), (A, S(1)), (C, R(1))\}$

... se desea realizar su representación gráfica y matricial.

El GAR correspondiente sería el siguiente:



Mientras que su representación matricial sería:

$$MA = \begin{array}{c|ccc} & R & S & T \\ \hline A & 1 & 0 & 1 \\ B & 1 & 1 & 0 \\ C & 0 & 1 & 0 \\ D & 1 & 0 & 1 \end{array}$$

$$MS = \begin{array}{c|ccc} & R & S & T \\ \hline A & 0 & 1 & 0 \\ B & 0 & 0 & 0 \\ C & 1 & 0 & 0 \\ D & 0 & 0 & 0 \end{array}$$

$$VR = \begin{array}{c|ccc} & R & S & T \\ \hline A & 3 & 2 & 2 \end{array}$$

$$VD = \begin{array}{c|ccc} & R & S & T \\ \hline A & 0 & 0 & 0 \end{array}$$

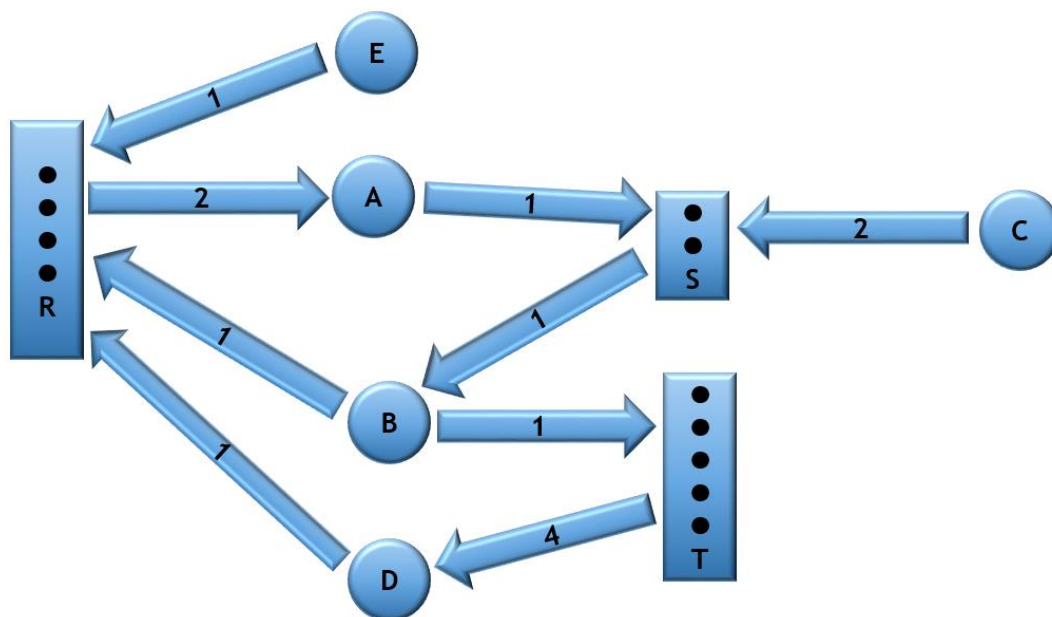
Ejemplo 2

Suponer que se desea realizar la representación mediante un GAR, y mediante una representación matricial, de los conjuntos:

$$N = \{A, B, C, D, E, R(4), S(2), T(5)\}$$

$$AR = \{(R(2), A), (T(4), D), (S(1), B), (B, R(1)), (C, S(2)), (A, S(1)), (D, R(1)), (E, R(1)), (B, T(1))\}$$

Entonces se tendría el siguiente grafo:



Y su representación matricial sería:

$$MA = \begin{array}{c|ccc} & R & S & T \\ \hline A & 2 & 0 & 0 \\ B & 0 & 1 & 0 \\ C & 0 & 0 & 0 \\ D & 0 & 0 & 4 \\ E & 0 & 0 & 0 \end{array}$$

$$MS = \begin{array}{c|ccc} & R & S & T \\ \hline A & 0 & 1 & 0 \\ B & 1 & 0 & 1 \\ C & 0 & 2 & 0 \\ D & 1 & 0 & 0 \\ E & 1 & 0 & 0 \end{array}$$

$$VR = \begin{array}{c|ccc} & R & S & T \\ \hline A & 4 & 2 & 5 \end{array}$$

$$VD = \begin{array}{c|ccc} & R & S & T \\ \hline A & 2 & 1 & 1 \end{array}$$

Estrategias ante un abrazo mortal

Existen cuatro tipos de estrategias utilizadas para tratar el abrazo mortal que son enumeradas a continuación:

Estrategias de prevención

Estas estrategias pueden ser indirectas y directas. Las primeras consisten en impedir la aparición de alguna de las tres primeras condiciones necesarias para la ocurrencia del abrazo mortal, mientras que las segundas buscan evitar la aparición de la espera circular que es imprescindible para la existencia de un abrazo mortal. Estos conceptos se traducen en las siguientes acciones a realizar ante cada una de las condiciones:

Exclusión mutua

La idea será la de definir recursos compartibles y no compartibles, para luego aplicar la exclusión mutua sólo a los que necesita. La objeción central a esta opción es que, por lo general, la exclusión mutua no puede anularse.

Espera y retención

Esta solución requiere que el proceso solicite todos los recursos que necesite a un mismo tiempo y que quede bloqueando hasta que todos los recursos puedan concedérsele simultáneamente. Esta opción resulta ineficiente por varios factores como ser: el desconocimiento que tiene el proceso de todos los recursos que necesitará, la posibilidad de que un proceso quede suspendido durante mucho tiempo, aunque podría continuar con algunos de los recursos disponibles, etc.

Sin desalojo (no expropiación)

Aunque la no apropiación puede prevenirse de varias formas, esta estrategia se usa sólo en recursos cuyo estado puede salvarse y restaurarse fácilmente más tarde. Una de las estrategias propone que cuando un proceso, que ya tiene asignados recursos, se le niegue una nueva asignación, ese proceso deberá liberar todos los recursos que tiene y luego pedirlos otra vez, junto con el recurso adicional, cuando los necesite. Otra variante, si el sistema trabaja con prioridades, consiste en expropiarle a un proceso los recursos que está utilizando y asignárselos al proceso con mayor prioridad que los está requiriendo. Esto evitará el abrazo mortal sólo si no son dos procesos que poseen la misma prioridad.

Espera circular

La estrategia empleada se denomina método de las peticiones ordenadas. Esta consiste en establecer un orden total de los recursos del sistema, por lo general creciente, que asegure el no bloqueo de los procesos, estableciendo la restricción de que un proceso debe pedir los recursos que necesita en el orden prefijado.

Estrategias de predicción

Estas estrategias plantean dos caminos diferentes para la predicción del interbloqueo. La primera de las estrategias, llamada negativa de iniciación de procesos, postula no iniciar un proceso si sus demandas pueden llevar a un abrazo mortal, mientras que la segunda

plantea no conceder una solicitud de incrementar los recursos de un proceso si esta asignación puede llevar a un abrazo mortal y se denomina negativa de asignación de recursos o algoritmo del banquero. En definitiva, para evitar el abrazo mortal será necesario que cada proceso declare el máximo número de recursos de cada tipo que puede llegar a necesitar y posteriormente implementar el método del estado seguro.

Ahora bien, la pregunta es, ¿cuándo el estado del sistema es seguro? Teniendo en claro que el estado del sistema es la asignación actual de recursos a los procesos, se puede afirmar que este se encuentra seguro cuando existe al menos un orden en el que todos los procesos se pueden ejecutar hasta su final sin generar un abrazo mortal. Cuando un proceso o un hilo requiera un recurso disponible, el sistema operativo debe verificar si la asignación del recurso solicitado no lleva al sistema a un estado inseguro, en caso de ser así debe cambiar el orden de ejecución de los procesos o hilos de manera tal que se mantenga un estado seguro. Obviamente, un estado inseguro es un estado que no es seguro en el que existe la posibilidad de abrazo mortal.

Negativa de iniciación de procesos

Es una estrategia muy deficiente porque asume que un proceso comenzará sólo si puede cumplirse su demanda máxima de recursos junto con la de todos los procesos actuales.

Negativa de asignación de recursos o algoritmo del banquero

La estrategia del algoritmo del banquero fue propuesta por primera vez por Dijkstra y presenta las características siguientes:

- Permite administrar múltiples tipos de recursos.
- Todos los procesos o hilos deben declarar el máximo de recursos que pueden usar.
- Cuando un proceso o hilo solicita un recurso quizás deba esperar.
- Después de que un proceso o hilo obtiene todos los recursos que necesita para ejecutar, los debe liberar dentro de una cantidad de tiempo finita.

Además, para verificar que los procesos están en estado seguro el sistema debe:

- Verificar que el requerimiento no supera lo declarado.
- Verificar que existan suficientes recursos disponibles.
- Simular la asignación, verificar si el sistema quedó en estado seguro y solo en caso afirmativo efectuar la asignación real.

Esta estrategia emplea las estructuras siguientes para n procesos y m recursos diferentes:

- Un vector de recursos de m elementos.
- Un vector de recursos disponibles de m elementos.
- Una matriz de demanda máxima de n * m elementos.
- Una matriz de asignaciones de n * m elementos.
- Una matriz de recursos solicitados, o de necesidad, de n * m elementos.

Con este conjunto de estructuras, el algoritmo del banquero deberá determinar si al conceder las unidades pedidas por un proceso, el sistema se mantiene o no en estado seguro. Si se mantiene entonces, como se dijo anteriormente, se conceden las unidades solicitadas, pero, si el estado pasa a ser inseguro, entonces se negará la asignación de unidades pedida.

Suponer entonces para describir el algoritmo que se tienen 5 procesos y 4 recursos diferentes. La planificación es FCFS y los procesos llegaron en orden alfabético a la cola de listos. En el estado actual, se han asignado recursos a los cinco procesos, quedando disponible 1 unidad de cada uno de los recursos. Ante la pregunta ¿es un estado seguro? se plantea otra intermedia ¿puede alguno de los cinco procesos ejecutarse completamente con los recursos disponibles? Es decir, ¿puede cubrirse la diferencia entre la asignación actual y la demanda máxima para cualquier proceso con los recursos disponibles?

														Vector de Recursos		6	4	3	3		
																R ₁	R ₂	R ₃	R ₄		
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad							
		Recursos (AR)				Recursos (DM)				Solicitados (DM-AR)											
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	1	1	VD ₀			
Procesos	A	0	1	0	0	2	2	1	2	2	1	1	2								
	B	3	0	1	1	4	1	1	1	1	1	0	0								
	C	1	1	1	0	4	2	1	0	3	1	0	0								
	D	1	1	0	1	1	1	1	1	0	0	1	0								
	E	0	0	0	0	2	1	1	0	2	1	1	0								

sus recursos pueden volver a la reserva de recursos disponibles. El estado resultante se muestra en la figura siguiente:

										Vector de Recursos				6	4	3	3	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad				
		Recursos (AR)				Recursos (DM)				Solicitados (DM-AR)				R ₁	R ₂	R ₃	R ₄	
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	1	1	VD ₀
Procesos	A	0	1	0	0	2	2	1	2	2	1	1	2					
	B	0	0	0	0	0	0	0	0	0	0	0	0	4	1	2	2	VD ₁
	C	1	1	1	0	4	2	1	0	3	1	0	0					
	D	1	1	0	1	1	1	1	1	0	0	1	0					
	E	0	0	0	0	2	1	1	0	2	1	1	0					

Ahora se puede preguntar de nuevo si alguno de los procesos restantes puede terminar. En este caso, cada uno de los procesos restantes puede completarse. Suponer que se elige A, se le asignan los recursos pedidos, finaliza A y todos sus recursos vuelven a la reserva disponible. Esto se puede apreciar en el estado representado por la figura siguiente:

										Vector de Recursos				6	4	3	3	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad				
		Recursos (AR)				Recursos (DM)				Solicitados (DM-AR)				R ₁	R ₂	R ₃	R ₄	
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	1	1	VD ₀
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	4	2	2	2	VD ₂
	B	0	0	0	0	0	0	0	0	0	0	0	0	4	1	2	2	VD ₁
	C	1	1	1	0	4	2	1	0	3	1	0	0					
	D	1	1	0	1	1	1	1	1	0	0	1	0					
	E	0	0	0	0	2	1	1	0	2	1	1	0					

A continuación, se puede completar C, finaliza C y todos sus recursos se suman a la reserva disponible dando como resultado el estado representado por la próxima figura:

			Vector de Recursos							
							6	4	3	3
							R ₁	R ₂	R ₃	R ₄
Asignaciones	Demanda	Recursos	Vector de							

		Recursos (AR)				Máxima Recursos (DM)				Solicitados (DM-AR)				disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	VD ₀
	B	0	0	0	0	0	0	0	0	0	0	0	0	4	2	2	2	VD ₂
	C	0	0	0	0	0	0	0	0	0	0	0	0	4	1	2	2	VD ₁
	D	1	1	0	1	1	1	1	1	0	0	1	0	5	3	3	2	VD ₃
	E	0	0	0	0	2	1	1	0	2	1	1	0					

Luego, se puede completar D. Nuevamente todos sus recursos se suman a la reserva disponible dando como resultado el estado representado por la próxima figura:

		Vector de Recursos																
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	VD ₀
	B	0	0	0	0	0	0	0	0	0	0	0	0	4	2	2	2	VD ₂
	C	0	0	0	0	0	0	0	0	0	0	0	0	4	1	2	2	VD ₁
	D	1	1	0	1	1	1	1	1	0	0	1	0	5	2	3	2	VD ₃
	E	0	0	0	0	2	1	1	0	2	1	1	0	6	4	3	3	VD ₄

Por último, se puede completar E.

Luego, aunque la secuencia inicial $A \Rightarrow B \Rightarrow C \Rightarrow D \Rightarrow E$ de asignación de recursos no concluía con un estado seguro, siguiendo la secuencia de asignación $B \Rightarrow A \Rightarrow C \Rightarrow D \Rightarrow E$ se tiene que todos los procesos han finalizado y, por lo tanto, el estado establecido por la figura inicial es un estado seguro.

Ahora bien, con el fin de cumplir con la planificación FCFS, este estado inicial seguro generará las siguientes estructuras.

		Vector de Recursos																
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
Asignaciones	Recursos (AR)	Demanda Máxima Recursos (DM)				Recursos Solicitados (DM-AR)				Vector de disponibilidad								
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	

		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	1	1	VD ₀
Procesos	B	3	0	1	1	4	1	1	1	1	1	0	0					
	A	0	1	0	0	2	2	1	2	2	1	1	2					
	C	1	1	1	0	4	2	1	0	3	1	0	0					
	D	1	1	0	1	1	1	1	1	0	0	1	0					
	E	0	0	0	0	2	1	1	0	2	1	1	0					

Dentro de todo este desarrollo e inadvertidamente, para conseguir este estado seguro, se implementó la siguiente estrategia:

- Cuando un proceso realiza una solicitud de un conjunto de recursos, se supone que la solicitud se concede, se actualiza el estado del sistema y se determina si el resultado es un estado seguro. Si lo es, se concede la solicitud; y, si no, se bloquea al proceso hasta que sea seguro conceder la solicitud.

Ahora bien, partiendo del estado representado por la siguiente figura:

										Vector de Recursos				6	4	3	3	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad				
		Recursos (AR)				Recursos (DM)				Solicitados (DM-AR)				R ₁	R ₂	R ₃	R ₄	
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	1	1	VD ₀
Procesos	A	0	1	0	0	2	2	1	2	2	1	1	2					
	B	3	0	1	1	4	1	1	1	1	1	0	0					
	C	1	1	1	0	4	2	1	0	3	1	0	0					
	D	1	1	0	1	1	1	1	1	0	0	1	0					
	E	0	0	0	0	2	1	1	0	2	1	1	0					

Y suponiendo que A realiza una solicitud de una unidad de R₂ y otra de R₃, ¿cuál sería el estado resultante? Admitiendo que se concede la solicitud, se tendría el estado mostrado por la siguiente figura:

														Vector de Recursos		6	4	3	3		
																R ₁	R ₂	R ₃	R ₄		
OC es A		Asignaciones Recursos (AR)				Demanda Máxima Recursos (DM)				Recursos Solicitados (DM-AR)				Vector de disponibilidad				VD ₀			
														R ₁	R ₂	R ₃	R ₄				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	0	0	1				
0	2	1	0	2	2	1	2	2	0	0	2										

		Recursos				Máxima Recursos				Solicitados				disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
		1	5	2	0	1	7	5	0	0	7	5	0	1	5	2	0	VD ₀
Procesos	A	0	0	1	2	0	0	1	2	0	0	0	0					
	B	1	0	0	0	1	7	5	0	0	7	5	0					
	C	1	3	5	4	2	3	5	6	1	0	0	2					
	D	0	6	3	2	0	6	5	2	0	0	2	0					
	E	0	0	1	4	0	6	5	6	0	6	4	4					

Con la disponibilidad actual se puede completar A hasta su finalización con lo cual todos sus recursos se acumulan en el vector de disponibilidad dando como resultado el estado representado por la próxima figura:

														Vector de Recursos				3	14	12	12	
														R ₁	R ₂	R ₃	R ₄					
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad								
		Recursos				Recursos				Solicitados				R ₁	R ₂	R ₃	R ₄					
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	5	2	0	VD ₀				
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	5	3	2	VD ₁				
	B	1	0	0	0	1	7	5	0	0	7	5	0									
	C	1	3	5	4	2	3	5	6	1	0	0	2									
	D	0	6	3	2	0	6	5	2	0	0	2	0									
	E	0	0	1	4	0	6	5	6	0	6	4	4									

Puesto que con la disponibilidad actual no es posible para completar B, a continuación, se puede completar C, por lo que finaliza C y todos sus recursos se suman al vector de disponibilidad dando como resultado el estado representado por la próxima figura:

														Vector de Recursos				3	14	12	12	
														R ₁	R ₂	R ₃	R ₄					
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad								
		Recursos				Recursos				Solicitados				R ₁	R ₂	R ₃	R ₄					
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	5	2	0	VD ₀				
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	5	3	2	VD ₁				
	B	1	0	0	0	1	7	5	0	0	7	5	0									
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	8	8	6	VD ₂				
	D	0	6	3	2	0	6	5	2	0	0	2	0									
	E	0	0	1	4	0	6	5	6	0	6	4	4									

Ahora, si se puede cumplir con la solicitud del proceso B, éste tendrá asignado el máximo de recursos necesarios y podrá ejecutarse hasta el final. Suponer que esto sucede. Cuando B termina, los recursos que le fueron asignados se pueden acumular en el vector de disponibilidad. El estado resultante se muestra en la figura siguiente:

														Vector de Recursos				3	14	12	12	
														R ₁	R ₂	R ₃	R ₄					
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad								
		Recursos				Recursos				Solicitados				R ₁	R ₂	R ₃	R ₄					
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	5	2	0	VD ₀				
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	5	3	2	VD ₁				
	B	0	0	0	0	0	0	0	0	0	0	0	0	3	8	8	6	VD ₃				
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	8	8	6	VD ₂				
	D	0	6	3	2	0	6	5	2	0	0	2	0									
	E	0	0	1	4	0	6	5	6	0	6	4	4									

Luego, se puede completar D. Nuevamente todos sus recursos se suman al vector de disponibilidad dando como resultado el estado representado por la próxima figura:

														Vector de Recursos				3	14	12	12	
														R ₁	R ₂	R ₃	R ₄					
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad								
		Recursos				Recursos				Solicitados				R ₁	R ₂	R ₃	R ₄					
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	5	2	0	VD ₀				
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	5	3	2	VD ₁				
	B	0	0	0	0	0	0	0	0	0	0	0	0	3	8	8	6	VD ₃				
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	8	8	6	VD ₂				
	D	0	0	0	0	0	0	0	0	0	0	0	0	3	14	11	8	VD ₄				
	E	0	0	1	4	0	6	5	6	0	6	4	4	⊗	⊗	⊗	⊗	⊗				

Por último, se puede completar E:

			Vector de Recursos				3	14	12	12	
							R ₁	R ₂	R ₃	R ₄	
Asignaciones Recursos	Demanda Máxima Recursos	Recursos Solicitados	Vector de disponibilidad								
			R ₁	R ₂	R ₃	R ₄					

		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	5	2	0	VD ₀
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	5	3	2	VD ₁
	B	0	0	0	0	0	0	0	0	0	0	0	0	3	8	8	6	VD ₃
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	8	8	6	VD ₂
	D	0	0	0	0	0	0	0	0	0	0	0	0	3	14	11	8	VD ₄
	E	0	0	0	0	0	0	0	0	0	0	0	0	3	14	12	12	VD ₅

Con lo cual se tiene una forma, $A \Rightarrow C \Rightarrow B \Rightarrow D \Rightarrow E$, de atender todas las solicitudes de recursos de los procesos y así poder concluirlos, aunque esa secuencia difiere del orden alfabético inicial de atención de las solicitudes. Por lo tanto, el estado actual es seguro.

Debe verse ahora si la asignación indicada en primer término puede realizarse. Si se asignan los recursos pedidos por B, (0, 4, 2, 0), se tiene el siguiente esquema en el cual se aplicará el algoritmo del banquero:

		Vector de Recursos												3	14	12	12	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones Recursos				Demanda Máxima Recursos				Recursos Solicitados				Vector de disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	0	0	VD ₀
Procesos	A	0	0	1	2	0	0	1	2	0	0	0	0					
	B	1	4	2	0	1	7	5	0	0	3	3	0					
	C	1	3	5	4	2	3	5	6	1	0	0	2					
	D	0	6	3	2	0	6	5	2	0	0	2	0					
	E	0	0	1	4	0	6	5	6	0	6	4	4					

Nuevamente, con la disponibilidad actual se puede completar A hasta su finalización con lo cual todos sus recursos se acumulan en el vector de disponibilidad dando como resultado el estado representado por la próxima figura:

		Vector de Recursos												3	14	12	12	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones Recursos				Demanda Máxima Recursos				Recursos Solicitados				Vector de disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	0	0	VD ₀
Proceso s	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	2	VD ₁
	B	1	4	2	0	1	7	5	0	0	3	3	0					

	C	1	3	5	4	2	3	5	6	1	0	0	2						
	D	0	6	3	2	0	6	5	2	0	0	2	0						
	E	0	0	1	4	0	6	5	6	0	6	4	4						

Con la disponibilidad actual se puede completar C, por lo que finaliza C y todos sus recursos se suman al vector de disponibilidad dando como resultado el estado representado por la próxima figura:

														Vector de Recursos				3	14	12	12	
														R ₁	R ₂	R ₃	R ₄					
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad								
		Recursos				Recursos				Solicitados				R ₁	R ₂	R ₃	R ₄					
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	0	0					
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	2	VD ₀				
	B	1	4	2	0	1	7	5	0	0	3	3	0									
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	4	6	6	VD ₂				
	D	0	6	3	2	0	6	5	2	0	0	2	0									
	E	0	0	1	4	0	6	5	6	0	6	4	4									

Luego, se puede completar D y seguidamente todos sus recursos se suman al vector de disponibilidad dando como resultado el estado representado por la próxima figura:

														Vector de Recursos				3	14	12	12	
														R ₁	R ₂	R ₃	R ₄					
		Asignaciones				Demanda Máxima				Recursos				Vector de disponibilidad								
		Recursos				Recursos				Solicitados				R ₁	R ₂	R ₃	R ₄					
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	0	0					
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	2	VD ₀				
	B	1	4	2	0	1	7	5	0	0	3	3	0									
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	4	6	6	VD ₂				
	D	0	0	0	0	0	0	0	0	0	0	0	0	2	10	9	8	VD ₃				
	E	0	0	1	4	0	6	5	6	0	6	4	4									

Con esta disponibilidad se puede completar E. Se lo completa y luego se acumulan los recursos que tenía asignados al vector de recursos disponibles y se deja al sistema en la siguiente situación:

		Vector de Recursos												3	14	12	12	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones Recursos				Demanda Máxima Recursos				Recursos Solicitados				Vector de disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
														1	1	0	0	VD ₀
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	2	VD ₁
	B	1	4	2	0	1	7	5	0	0	3	3	0					
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	4	6	6	VD ₂
	D	0	0	0	0	0	0	0	0	0	0	0	0	2	10	9	8	VD ₃
	E	0	0	0	0	0	0	0	0	0	0	0	0	2	10	10	12	VD ₄

Finalmente, se puede completar B:

		Vector de Recursos												3	14	12	12	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones Recursos				Demanda Máxima Recursos				Recursos Solicitados				Vector de disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	
														1	1	0	0	VD ₀
Procesos	A	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	2	VD ₁
	B	0	0	0	0	0	0	0	0	0	0	0	0	3	14	12	12	VD ₅
	C	0	0	0	0	0	0	0	0	0	0	0	0	2	4	6	6	VD ₂
	D	0	0	0	0	0	0	0	0	0	0	0	0	2	10	9	8	VD ₃
	E	0	0	0	0	0	0	0	0	0	0	0	0	2	10	10	12	VD ₄

Por lo tanto, se tiene una forma, $A \Rightarrow C \Rightarrow D \Rightarrow E \Rightarrow B$, de atender todas las solicitudes de recursos de los procesos y así poder concluirlos. Por lo tanto, el estado actual es seguro. Entonces el sistema operativo le asigna al proceso B los recursos que había solicitado quedando el sistema como se puede apreciar en el siguiente esquema:

		Vector de Recursos												3	14	12	12	
														R ₁	R ₂	R ₃	R ₄	
		Asignaciones Recursos				Demanda Máxima Recursos				Recursos Solicitados				Vector de disponibilidad				
		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	

		R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	1	1	0	0	VD ₀
Procesos	A	0	0	1	2	0	0	1	2	0	0	0	0					
	B	1	4	2	0	1	7	5	0	0	3	3	0					
	C	1	3	5	4	2	3	5	6	1	0	0	2					
	D	0	6	3	2	0	6	5	2	0	0	2	0					
	E	0	0	1	4	0	6	5	6	0	6	4	4					

Ahora la cuestión es si el proceso E solicita la asignación (0, 2, 4, 0), ¿se puede atender de inmediato dicha solicitud? La respuesta es negativa porque sólo se tiene una unidad de R₂ y no existen unidades disponibles de R₃. Luego la solicitud se deniega y entonces el proceso E se bloquea hasta que estén disponibles las unidades solicitadas y se pueda atender nuevamente su solicitud.

Estrategias de detección y recuperación

Esta estrategia de tratamiento del abrazo mortal permite el bloqueo, pero lo detecta y recupera el sistema. Presenta, como su nombre indica, dos fases: una fase de detección y otra fase de recuperación.

Fase de detección

En esta etapa debe ejecutarse un algoritmo que determine si el estado actual del sistema está libre de interbloqueos y que, en caso de que no lo esté, identifique qué procesos están implicados en el abrazo mortal para luego iniciar la fase de recuperación del sistema.

Uno de los tantos algoritmos usados para la detección del abrazo mortal es el que propusieron E. Coffman, M. Elphick y A. Shoshani en su trabajo *System Deadlocks* de junio de 1971. El algoritmo utiliza, para un sistema con n procesos y m recursos diferentes, una matriz de asignación Ar de n * m elementos, una matriz de solicitudes S, también de n * m elementos, y un vector de recursos disponibles VD de m componentes. Su desarrollo es el siguiente:

- 1) Se marca cada proceso que tenga una fila de la matriz Ar completamente en cero.
- 2) Se inicializa un vector temporal T con los valores del vector VD.
- 3) Se busca un índice I tal que el proceso I no esté actualmente marcado y la I-ésima fila de S sea menor o igual que T, componente a componente. Es decir, $S_{IK} \leq T_K$ para

$1 \leq K \leq m$, siendo m la cantidad de recursos diferentes. Si no se encuentra ninguna fila, el algoritmo termina.

- 4) Si se encuentra una fila que cumpla con la condición del paso anterior, se marca al proceso I y se suma la fila I de la matriz Ar al vector T . Es decir, se hace $T_K = T_K + Ar_{IK}$ con $1 \leq K \leq m$. A continuación, se vuelve al tercer paso.

Si y sólo si hay procesos sin marcar al final del algoritmo se puede afirmar que entre estos procesos existe un abrazo mortal. En tal caso el sistema pasará a la fase de recuperación.

Ejemplo 1

Se tiene el vector de disponibilidad y las matrices Ar y S :

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A	3	0	1	1
	B	0	1	1	1
	C	1	1	1	0
	D	1	1	0	1
	E	0	0	0	0
	F	4	2	0	3
	G	1	5	1	0
	H	3	2	2	1

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A	1	1	1	1
	B	0	2	1	2
	C	3	1	2	7
	D	0	0	4	0
	E	2	1	1	5
	F	2	1	0	2
	G	0	5	0	0
	H	1	0	4	2

Vector de disponibilidad				
	2	1	1	1
	R ₁	R ₂	R ₃	R ₄

Al aplicar el algoritmo se obtiene:

Asignaciones de recursos (Ar)						Recursos Solicitados (S)						Vector de disponibilidad					
Procesos	Recursos					Procesos	Recursos										
		R ₁	R ₂	R ₃	R ₄			R ₁	R ₂	R ₃	R ₄						
	A*	3	0	1	1		A	1	1	1	1						
	B	0	1	1	1		B	0	2	1	2						
	C	1	1	1	0		C	3	1	2	7						
	D	1	1	0	1		D	0	0	4	0						
E*	0	0	0	0	E	2	1	1	5								
F	4	2	0	3	F	2	1	0	2								
G	1	5	1	0	G	0	5	0	0								
H	3	2	2	1	H	1	0	4	2								

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*	3	0	1	1
	B	0	1	1	1
	C	1	1	1	0
	D	1	1	0	1
	E*	0	0	0	0
	F*	4	2	0	3
	G	1	5	1	0
	H	3	2	2	1

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A	1	1	1	1
	B	0	2	1	2
	C	3	1	2	7
	D	0	0	4	0
	E	2	1	1	5
	F	2	1	0	2
	G	0	5	0	0
	H	1	0	4	2

Vector de disponibilidad					
2	1	1	1	T ₀	
R ₁	R ₂	R ₃	R ₄		
5	1	2	2	T ₁	

Asignaciones de recursos (Ar)						Recursos Solicitados (S)						Vector de disponibilidad					
Procesos	Recursos					Procesos	Recursos										
		R ₁	R ₂	R ₃	R ₄			R ₁	R ₂	R ₃	R ₄						
	A*	3	0	1	1		A	1	1	1	1		2	1	1	T ₀	
	B*	0	1	1	1		B	0	2	1	2		5	1	2	T ₁	
	C	1	1	1	0		C	3	1	2	7		9	4	3	T ₃	
	D	1	1	0	1		D	0	0	4	0						
	E*	0	0	0	0		E	2	1	1	5						
	F*	4	2	0	3		F	2	1	0	2						
	G	1	5	1	0		G	0	5	0	0						
	H	3	2	2	1		H	1	0	4	2						

En este momento se comprueba que el algoritmo no puede continuar porque con la última disponibilidad (9, 4, 3,6) no es posible satisfacer las necesidades de los procesos C, D, G y H, los que no pueden ser marcados. Luego estos cuatro procesos se encuentran en un abrazo mortal.

En su libro *Sistemas operativos modernos* (Pearson / Prentice Hall, 2009), Andrew S. Tanenbaum plantea una variante de este algoritmo que es la siguiente:

- 1) Buscar un proceso desmarcado, P_i , para el que la i -ésima fila de S sea menor o igual que T .
- 2) Si se encuentra dicho proceso, se suma la i -ésima fila de A_r al vector T , se marca el proceso y se regresa al paso 1.
- 3) Si no existe dicho proceso, el algoritmo termina.

Ejemplo 2

Se tiene el siguiente sistema:

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A	2	0	0	0
	B	0	1	0	1
	C	0	2	0	1
	D	1	0	0	1
	E	0	0	1	0

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A	2	1	0	0
	B	2	2	1	0
	C	1	2	1	2
	D	3	1	0	1
	E	0	0	2	0

Vector de disponibilidad				
2	1	1	0	T ₀
R ₁	R ₂	R ₃	R ₄	

Al aplicar el algoritmo de Tanenbaum, con la disponibilidad inicial planteada, sólo se puede cumplir con las solicitudes del proceso A, luego se acumula y se obtiene:

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B	0	1	0	1
	C	0	2	0	1
	D	1	0	0	1
	E	0	0	1	0

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B	2	2	1	0
	C	1	2	1	2
	D	3	1	0	1
	E	0	0	2	0

Vector de disponibilidad				
2	1	1	0	T ₀
R ₁	R ₂	R ₃	R ₄	
4	1	1	0	T ₁

Con la nueva disponibilidad T₁ no es posible satisfacer las solicitudes de los procesos B, C, D y E por lo que estos procesos se bloquean, quedando entonces en un abrazo mortal.

Ejemplo 3

Dado el siguiente esquema se quiere determinar si el sistema está en un abrazo mortal:

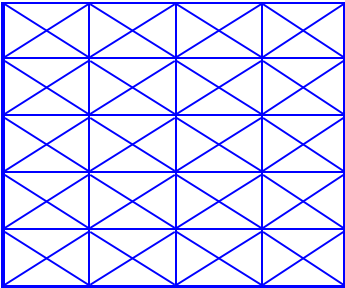
Asignaciones de recursos (Ar)			
	Recursos		
	R ₁	R ₂	R ₃

Recursos Solicitados (S)			
	Recursos		
	R ₁	R ₂	R ₃

Vector de disponibilidad			
0	1	0	T ₀
R ₁	R ₂	R ₃	

Procesos	A	0	2	0
	B	2	0	0
	C	1	0	1
	D	2	1	1
	E	0	0	2

Procesos	A	0	1	0
	B	2	0	2
	C	2	0	1
	D	2	2	1
	E	1	0	2



Con la disponibilidad T_0 se puede completar A y se obtiene el nuevo estado:

Asignaciones de recursos (Ar)					Recursos Solicitados (S)					Vector de disponibilidad				
Procesos	Recursos				Recursos				0	1	0	T ₀		
		R ₁	R ₂	R ₃		R ₁	R ₂	R ₃	R ₁	R ₂	R ₃			
	A*				A*				0	3	0	T ₁		
	B	2	0	0	B	2	0	2						
	C	1	0	1	C	2	0	1						
	D	2	1	1	D	2	2	1						
	E	2	0	0	E	0	1	0						

Luego, se acumulan en T_0 los recursos asignados al proceso A y se logra la disponibilidad T_1 que permitirá cubrir sólo la solicitud del proceso E, obteniéndose:

Asignaciones de recursos (Ar)					Recursos Solicitados (S)					Vector de disponibilidad				
Procesos	Recursos				Procesos	Recursos				0	1	0	T ₀	
		R ₁	R ₂	R ₃			R ₁	R ₂	R ₃	R ₁	R ₂	R ₃		
	A*					A*				0	3	0	T ₁	
	B	2	0	0		B	2	0	2					
	C	1	0	1		C	2	0	1					
	D	2	1	1		D	2	2	1					
	E*					E*				2	3	0	T ₂	

Con la disponibilidad T_2 no resulta posible cumplir con las demandas de B, C o D porque no existen unidades de R_3 disponibles, por lo cual estos 3 procesos se bloquean y quedan en un abrazo mortal.

El proceso C es el que tiene la mayor cantidad de recursos asignados. Luego de la expropiación se suman sus recursos al vector de disponibilidad actual (T_1) obteniéndose la nueva disponibilidad (T_2) y entonces se tiene el siguiente estado:

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B	0	1	0	1
	C	0	0	0	0
	D	1	0	0	1
E	0	0	1	0	

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B	2	2	1	0
	C	1	4	1	3
	D	3	1	0	1
E	0	0	2	0	

Vector de disponibilidad					
4	1	1	0	T ₁	
R ₁	R ₂	R ₃	R ₄		

Con esta nueva disponibilidad resta por ver si el sistema ya no tiene un abrazo mortal. En este momento es posible satisfacer las demandas del proceso B y entonces el estado sería el siguiente:

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B*				
	C	0	0	0	0
	D	1	0	0	1
	E	0	0	1	0

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B*				
	C	1	4	1	3
	D	3	1	0	1
	E	0	0	2	0

Vector de disponibilidad					
4	3	1	1	T ₂	
R ₁	R ₂	R ₃	R ₄		
4	4	1	2	T ₃	

Ahora, con la disponibilidad T_3 se puede satisfacer la solicitud de D por lo que se acumulan sus asignaciones al vector de disponibilidad quedando:

Asignaciones de recursos (Ar)					
Recursos					
ces					
A*					
R1R2R3R4					

Recursos Solicitados (S)					
Recursos					
ces					
A*					
R1R2R3R4					

Vector de disponibilidad					
4311T2					
R1R2R3R4					

	B*				
	C	0	0	0	0
	D*				
	E	0	0	1	0

	B*				
	C	1	4	1	3
	D*				
	E	0	0	2	0

4	4	1	2	T₃
5	4	1	3	T₄

Se obtiene la disponibilidad T4 con la cual se puede satisfacer la solicitud de C y entonces el esquema sería:

Asignaciones de recursos (Ar)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B*				
	C*				
	D*				
	E	0	0	1	0

Recursos Solicitados (S)					
Procesos	Recursos				
		R ₁	R ₂	R ₃	R ₄
	A*				
	B*				
	C*				
	D*				
	E	0	0	2	0

Vector de disponibilidad				
4	3	1	1	T₂
R ₁	R ₂	R ₃	R ₄	
4	4	1	2	T₃
5	4	1	3	T₅
5	4	1	3	T₄

Finalmente se puede cumplir con la solicitud de E y por lo tanto se comprueba que el sistema no está en abrazo mortal.

Ignorar el abrazo mortal o política del avestruz

Esta estrategia parte de la idea que, ante la cantidad de condiciones y / o restricciones que se presentan en las estrategias de prevención, de predicción o de detección y recuperación resulta más conveniente ignorar simplemente la ocurrencia del abrazo mortal. Pero, ignorar el problema puede tener como consecuencia que, cuando se produce un abrazo mortal, los procesos implicados seguirán bloqueados indefinidamente y los recursos que ellos tienen asignados tampoco podrán ser utilizados por otros procesos que no están bloqueados.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.